



Sécurité des systèmes

Cours 4 – Shellcoding & memory
permissions



- **The worst vulnerabilities can be exploited to gain arbitrary code execution (ACE)**
 - They allow running malicious code in the context of the target
 - Or even Remote code execution (RCE) when the vuln can be exploited remotely
 - Stack buffer overflows typical vulnerabilities that can cause arbitrary code execution
- **In this chapter we'll focus on the code that is executed**

Payloads

3



- **We call this code a payload**

- The role of the software exploit is to force the target to execute the payload

- **The payload is a malicious piece of code used by the attacker to achieve its goal.**

- Obtaining a remote shell on the device
- Installing a persisting software implant
- Etc.



- **The payload can be written in any programming language**
 - Chosen by the attacker depending on the targetted system
 - On Linux, it might be a simple shell script
 - On Windows, it could also be a powershell script
 - It can also be a compiled binary, or any kind of program
- **Exploit chains can use several payloads, and several exploits, e.g.**
 - An exploit calls `system()` to run a small command (first payload)
 - The shell command downloads and runs a second payload
 - The second payload runs an exploit to gain kernel privileges
 - A third payload is executed with kernel privileges



- **Many software exploits aims at running a *shellcode***
 - A simple piece of assembly code that is introduced in the program
 - Usually used to provide a shell (that's why its called a shellcode)
 - But can also suit different needs
 - For instance, a *stager* aims at loading a second stage of the payload
 - Why not just call `system()` ?
 - Sometimes there is no shell on the device (e.g. embedded devices)
 - Sometimes there is a sandbox, and calling `system()` is forbidden
 - The shellcode might then be used to perform privilege elevation

Shellcoding

6



- **How to introduce our own code in the target process ?**
 - Assembly instructions are just bytes
 - A shellcode is just some compiled code added in the program
 - Executed by gaining control of the instruction pointer

```
$ objdump -d exo1
[...]
00001170 <_start>:
    1170: 31 ed          xor    %ebp,%ebp
    1172: 5e            pop    %esi
[...]
```

■ Simple shellcode :

- `write(1, "hello\n", 6)`
- `exit()`

■ x86 syscalls

- Triggered by `int $0x80`
- Syscall number in `eax`
- Arguments in `ebx`, `ecx`, `edx`...

■ Jmp/call trick

- Used to resolve address of the message



```
1  _start:
2  // syscall(write:4, fd=1, buf="hello world!", 13)
3  mov $4, %eax
4  mov $1, %ebx
5  jmp buf
6  next:
7  pop %ecx
8  mov $6, %edx
9  int $0x80
10 // syscall(exit:1)
11 mov $1, %eax
12 int $0x80
13 buf:
14 call next
15 .asciz "hello\n"
```

Shellcoding

8



■ Disassembled shellcode

```
00001000 <_start>:  
    1000: b8 04 00 00 00      mov    $0x4,%eax  
    1005: bb 01 00 00 00      mov    $0x1,%ebx  
    100a: eb 0f                jmp     101b <msg>  
[...]
```

■ Full extracted shellcode

```
00000000 b8 04 00 00 00 bb 01 00 00 00 eb 0f 59 ba 06 00 |.....Y...|  
00000010 00 00 cd 80 b8 01 00 00 00 cd 80 e8 ec ff ff |.....|  
00000020 68 65 6c 6c 6f 0a 00 |hello..|
```

Constraints

9



- **The shellcode must be in an executable page**

... which is uncommon on modern systems

```
$ cat /proc/$(pidof exo1)/maps
565c8000-565c9000 r--p 00000000 fe:01 44827465 exo1
565c9000-565ca000 r-xp 00001000 fe:01 44827465 exo1
565ca000-565cb000 r--p 00002000 fe:01 44827465 exo1
565cb000-565cc000 r--p 00002000 fe:01 44827465 exo1
565cc000-565cd000 rw-p 00003000 fe:01 44827465 exo1
```

- Most programs do not contain any rwx section
- **An exploit could call mprotect() to make a page executable**
 - Extra steps..



■ Silly example (lets imagine this runs on Linux)

```
void run_shellcode(void *buf, size_t size)
{
    void (*code)(void) = malloc(size) ;
    memcpy(code, buf, size) ;
    code() ;
}
```

■ This will cause a segmentation fault

- Because the memory returned by malloc() is not executable



■ Fixed example

```
void run_shellcode(void *buf, size_t size)
{
    void (*code)(void) = malloc(size);
    memcpy(code, buf, size);
    mprotect(code &~(PAGE_SIZE-1),
              PAGE_SIZE,
              PROT_READ|PROT_WRITE|PROT_EXEC);
    code();
}
```

■ mprotect() changes the protection on a memory page

- cf. man mprotect



■ There can be more constraints

- Depending on where/how the shellcode is implanted
- Depending on how its processed by the program
- For instance, if processed as a string
 - Must not contain NULL-bytes to avoid truncation by string copies
 - Might require using a constrained set of characters
 - ie. Alphanumeric only, no carriage return, etc.
- The size is often constrained
 - For instance, if stored in a fixed-size buffer



■ Bypassing constraints

- Usually done by *staging* the payload
- A first shellcode will load a secondary payload
 - The first shellcode will be small and respect all constraints
 - It's role is to retrieve the second stage
 - By downloading it
 - By finding it elsewhere in memory (egg hunter)
 - Etc...
 - Then load/execute the second stage
 - The second stage will be the actual payload
 - It has less constraints on it
 - Can be written in a higher-level language (e.g. C)



■ Exemple : shellcode to print "hello"

- It contains many zero bytes, and can't be copied with strcpy()

```
00001000 <_start>:
    1000: b8 04 00 00 00      mov     $0x4,%eax
    1005: bb 01 00 00 00      mov     $0x1,%ebx
    100a: eb 0f              jmp     101b <msg>

0000100c <next>:
    100c: 59                pop     %ecx
    100d: ba 06 00 00 00      mov     $0x6,%edx
    1012: cd 80              int     $0x80
    [...]

```



- We can rewrite it with different instructions, with no zeros

```
00001000 <_start>:
    1000: 31 db          xor    %ebx,%ebx
    1002: 43          inc    %ebx
    1003: 8d 43 03     lea    0x3(%ebx),%eax
    1006: 8d 53 05     lea    0x5(%ebx),%edx
    1009: eb 09       jmp    1014 <msg>

0000100b <next>:
    100b: 59          pop    %ecx
    100c: cd 80       int    $0x80
    [...]
```



<https://www.linkedin.com/company/synacktiv>

<https://twitter.com/synacktiv>

Nos publications sur : <https://synacktiv.com>